

UCS503 - Software Engineering

Collaborative Study Platform

A Web-Based Academic Resource-Sharing Platform for Students

Submitted by:

1024170430 Anirudh

1024170449 Shashvt

1024170441 Lavish

Group: 5, BE Third Year, CSE

Submitted to:

Ms Nisha Thakur

Mid-Semester Evaluation

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

Chapter 1 Introduction.....3

1.1 Background and Context.....3

1.1.1 Problem Statement..... 3

1.2 Project Objectives..... 3

Chapter 2 Methodology and Design.....4

2.1 System Architecture.....4

2.1.1 High-Level Design.....4

2.1.2 Technical Stack.....8

2.2 Implementation Details.....8

2.2.1 Module 1: Core Functionality.....8

Chapter 3 Progress Evaluation.....9

3.1 Current Status and Validation.....9

3.2 Analysis of Progress.....9

Chapter 1 Introduction

1.1 Background and Context

The Collaborative Study Platform is a web-based academic resource-sharing platform designed primarily for students. Its purpose is to provide students with a centralized place to access and share academic study material. The platform organizes academic resources according to the student's academic context, and the major resources available through the platform are Study Notes, Books, and Previous Year Questions (PYQs). The platform also provides functionality for uploading study material so that academic resources can be added to the platform.

1.1.1 Problem Statement

Students currently require a centralized place to access and share academic study material relevant to their specific academic context. The platform addresses this by organizing resources according to classroom, year/stream, and subject, and by allowing users to both retrieve existing resources (notes, books, PYQs) and contribute new study material through an upload facility.

1.2 Project Objectives

- To provide a centralized platform where students can access academic Study Notes, Books, and Previous Year Questions (PYQs), organized according to their academic context (classroom, year/stream, and subject).
- To implement a secure user authentication system (registration and login) that protects the platform's user-specific functionality.
- To allow authenticated students to navigate a dashboard, select their academic context (classroom, year/stream, subject), and view or download the corresponding notes, books, and PYQs.
- To provide functionality for students to upload study material, including entry of relevant metadata and selection of a file, so that additional academic resources can be added to the platform.

Chapter 2 Methodology and Design

2.1 System Architecture

The platform follows a client-server architecture. The Student/User interacts with the React Frontend, which communicates with the Go/Gin Backend over REST-style APIs. The backend, in turn, communicates with PostgreSQL and AWS S3 where required. The user does not directly communicate with PostgreSQL or AWS S3. Uploaded files are stored in AWS S3, while the information describing each file (filename, year, subject code, stream, uploader information, and the AWS S3 reference/key) is stored in PostgreSQL.

2.1.1 High-Level Design

The general system workflow proceeds as follows: user authentication, followed by academic context selection (classroom, then year/stream, then subject), followed by study resource access (notes, books, or PYQs) or, optionally, study material upload, after which the user may continue using the platform or log out. The diagrams below illustrate the actors, workflows, and interactions that make up this design.

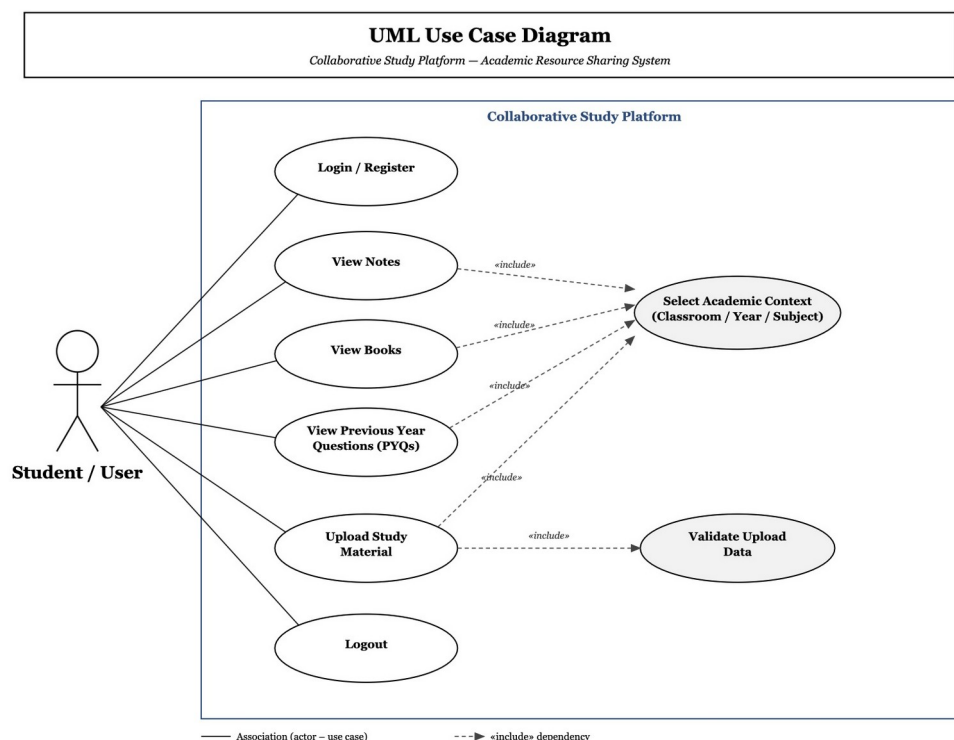


Figure 2.1: UML Use Case Diagram – Collaborative Study Platform

The Student/User is the sole actor of the system and can Login/Register, View Notes, View Books, View Previous Year Questions, Upload Study Material, and Logout. The use cases View Notes, View Books, View Previous Year Questions, and Upload Study Material each include the Select Academic Context use case, and Upload Study Material additionally includes Validate Upload Data.

Sample Use Case – UC-01: User Login

1. Use Case Title	User Login
2. Abbreviated Title	Login
3. Use Case Id	UC-01
4. Actors	Student/User
5. Description Allows a registered user to authenticate and gain access to the Collaborative Study Platform using their registered credentials.	
5.1. Pre Conditions: User must have a registered account.	
5.2. Task Sequence 1. User opens the login screen. 2. User enters email and password. 3. React Frontend sends the credentials to the Go/Gin Backend for verification. 4. Backend validates credentials against PostgreSQL and returns a session token.	
5.3. Post Conditions: 1. User is authenticated and redirected to the Dashboard. 2. Session is stored for subsequent requests.	
6. Modification History: Date 17-Sep-2026	
7. Author: Anirudh	

Table 2.1: Sample Use Case Description – UC-01 User Login

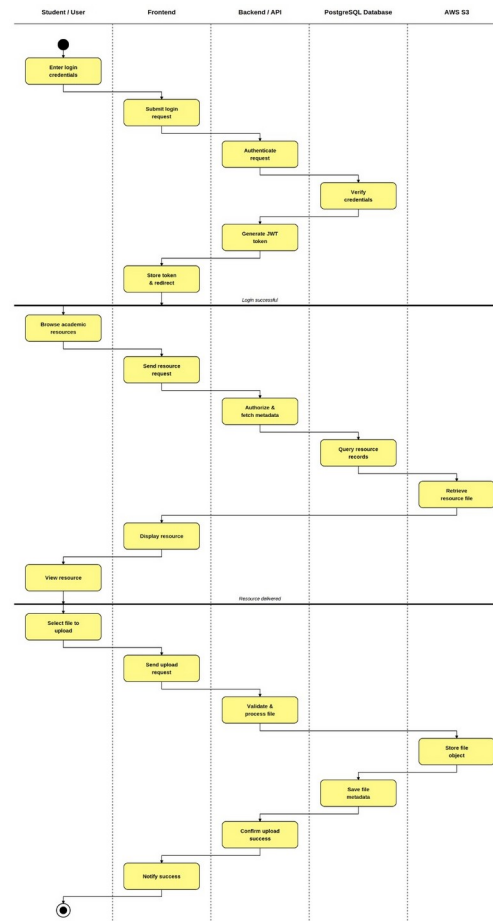


Figure 2.2: Activity Diagram – Login, Resource Access, and Upload Workflow

This activity diagram traces the complete runtime workflow across three phases separated by synchronization bars: authentication (entering credentials through token storage and redirect), resource access (browsing academic resources through display of the requested resource), and study material upload (selecting a file through confirmation and notification of upload success).

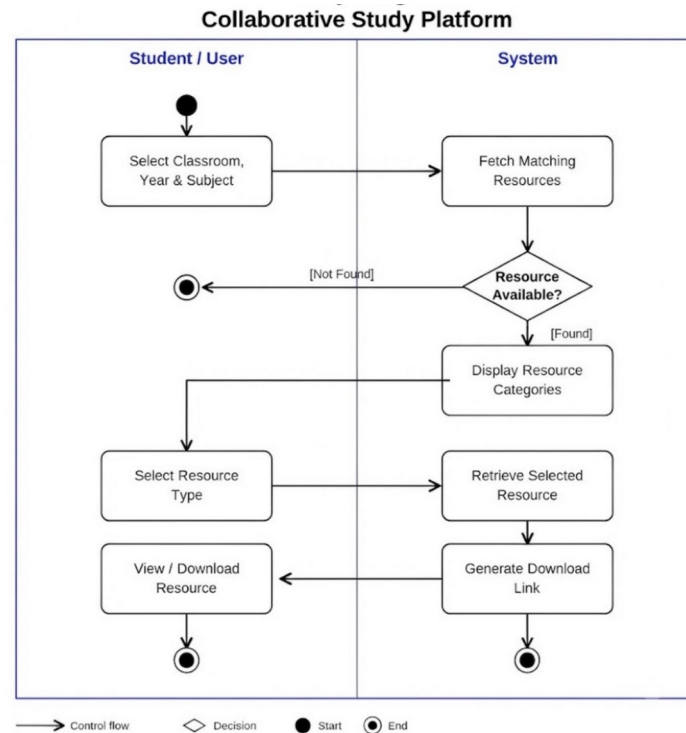


Figure 2.3: Swimlane Diagram – Resource Access Flow

This diagram details the resource-access flow specifically: after the user selects a classroom, year, and subject, the system fetches matching resources and checks whether a resource is available. If not found, the flow ends; if found, the system displays resource categories, the user selects a resource type, and the system retrieves the resource and generates a download link for the user to view or download.

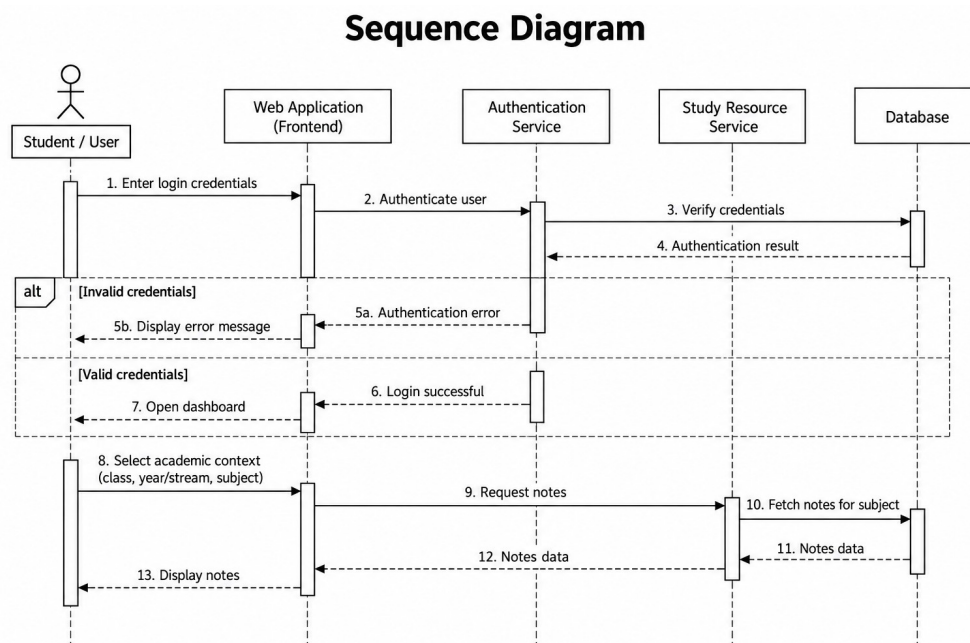


Figure 2.4: Sequence Diagram – Login and Notes Retrieval

The sequence diagram shows the message exchange between the Student/User, the Web Application (Frontend), the Authentication Service, the Study Resource Service, and the Database for the login flow (including the invalid- and valid-credentials alternatives) followed by the notes-retrieval flow after academic context selection.

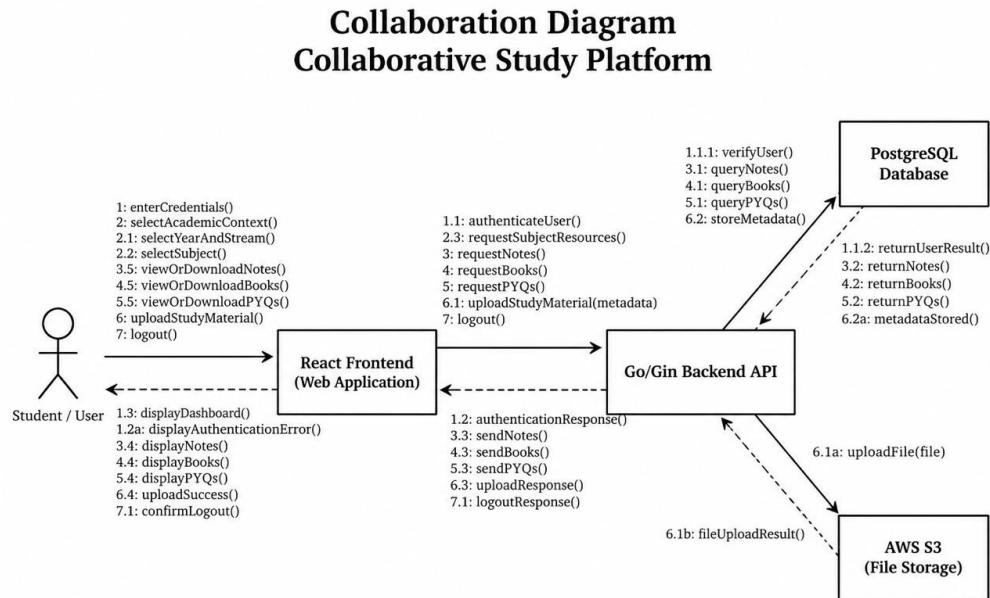


Figure 2.5: Collaboration Diagram – Collaborative Study Platform

The collaboration diagram illustrates the same set of interactions as the sequence diagram from an object-collaboration perspective, showing numbered messages exchanged between the Student/User, the React Frontend, the Go/Gin Backend API, the PostgreSQL Database, and AWS S3 for authentication, resource retrieval, study material upload, and logout.

2.1.2 Technical Stack

- Frontend: React, TypeScript / TSX, Axios, Tailwind CSS
- Backend: Go, Gin framework, REST-style APIs
- Database: PostgreSQL
- File Storage: AWS S3

2.2 Implementation Details

2.2.1 Module 1: Core Functionality – Authentication and Academic Navigation

The authentication module allows a Student/User to Login or Register. The Frontend sends an authentication request to the Backend, which verifies the user's credentials. If the credentials are invalid, an authentication error is displayed and the user returns to the Login/Register screen; if valid, the Dashboard is opened. After successful authentication, the user reaches the Dashboard and selects their academic context in sequence: Classroom/Academic Context, then Year/Stream, then Subject, before choosing a study action (View Notes, View/Download Books, View/Download PYQs, or Upload Study Material).

Chapter 3 Progress Evaluation

3.1 Current Status and Validation

Following the design described in Chapter 2, a working frontend prototype of the Collaborative Study Platform has been implemented and deployed locally for testing. The prototype validates the User Login use case (UC-01, Table 2.1) and the authentication portion of the activity, sequence, and collaboration diagrams in Section 2.1.1: a student can register a new account or sign in with existing credentials, and on success is taken to a personalised dashboard.

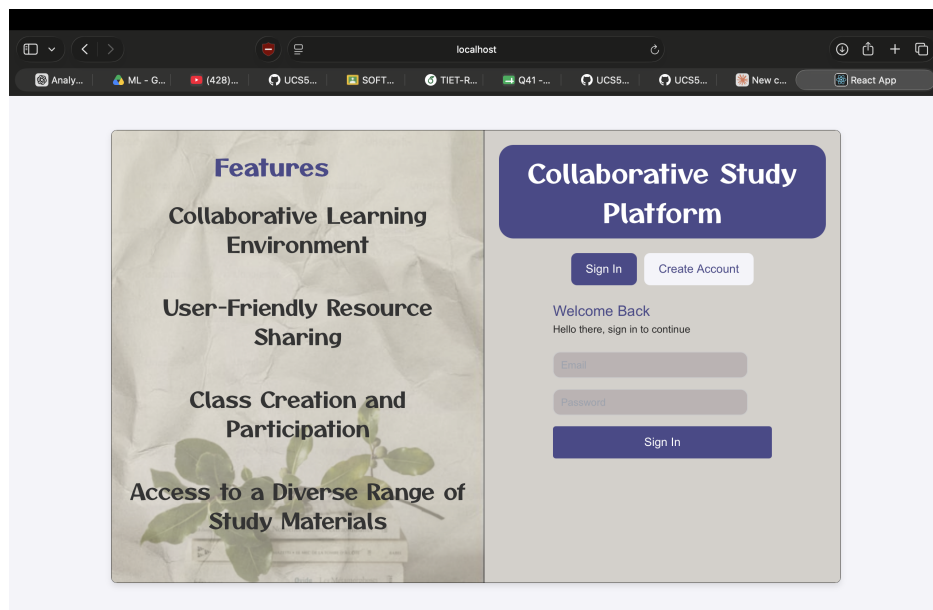


Figure 3.1: Landing / authentication screen of the Collaborative Study Platform prototype, showing the feature highlights alongside the Sign In and Create Account forms.

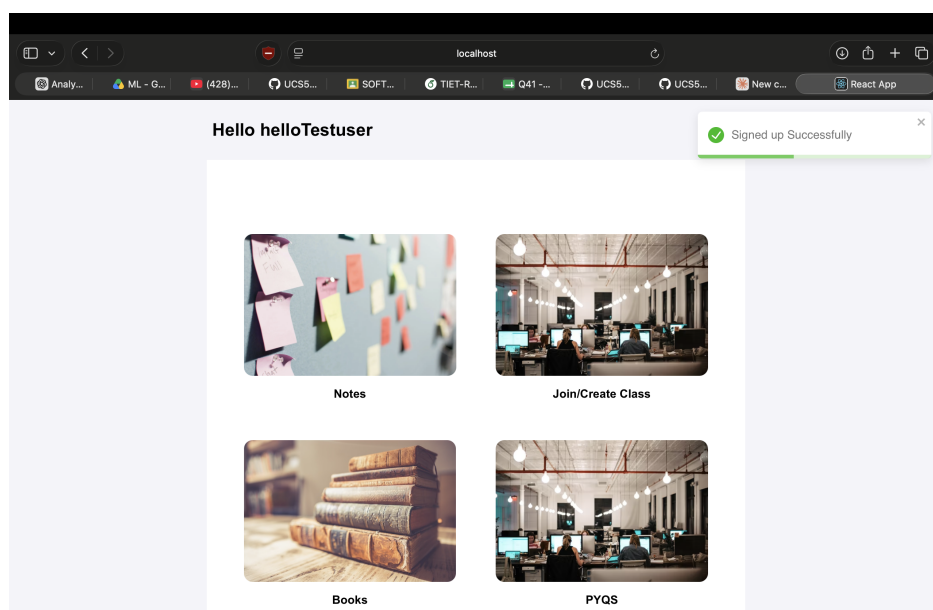


Figure 3.2: Dashboard displayed immediately after a successful registration, confirming the authentication flow and exposing the Notes, Join/Create Class, Books, and PYQs modules to the

user.

As shown in Figure 3.2, registration returns a success confirmation and routes the user directly to the Dashboard, consistent with the post-conditions listed in Table 2.1 and with steps 6-7 of the Sequence Diagram in Figure 2.4. The four dashboard cards correspond to the View Notes, View Books, View Previous Year Questions, and Join/Create Class use cases identified in the UML Use Case Diagram (Figure 2.1).

3.2 Analysis of Progress

Measured against the four project objectives set out in Section 1.2, progress to date is as follows:

- **Centralized resource access (Objective 1):** the Dashboard now exposes Notes, Books, PYQs, and Join/Create Class as distinct modules; the screens are built, and wiring them to live academic-context data is the next step.
- **Secure authentication (Objective 2):** largely complete. Registration and sign-in screens are functional end-to-end against the Go/Gin backend, and a successful sign-up correctly returns a confirmation and redirects to the Dashboard, as captured in Figure 3.2.
- **Academic-context navigation (Objective 3):** the Dashboard and its four resource cards are in place; the classroom → year/stream → subject selection flow from the Swimlane Diagram (Figure 2.3) is designed but not yet connected to these screens.
- **Study-material upload (Objective 4):** not yet started on the frontend; the corresponding backend upload and validation endpoints described in Section 2.1 are planned for the next development cycle.

Overall, the authentication module and the dashboard shell are functionally complete and match the UML use case, sequence, and collaboration diagrams presented in Chapter 2. The remaining work is concentrated in the resource-retrieval and upload modules, both of which are architecturally defined but still to be implemented on the frontend.